指导书

实验一、认识数据与数据预处理

1.1 安装并配置 JAVA，Eclipse 和 WEKA

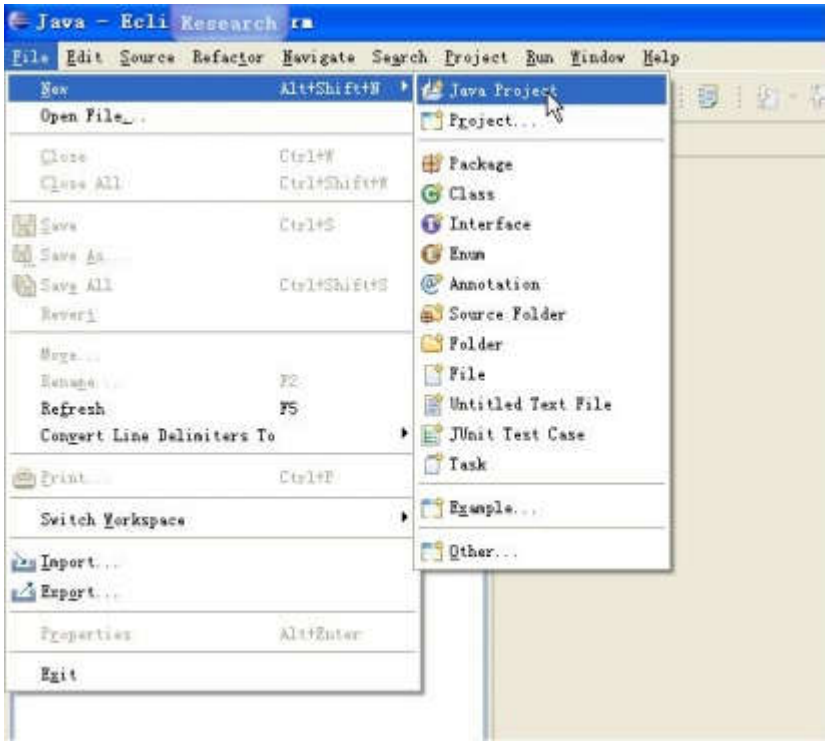
(1) 检查 JAVA 是否正确安装和配置 （在命令行 cmd 中输入：java -version）
(2) Eclipse 平台（无需任何配置）
(3) 安装 WEKA

1.2 如何使用 WEKA 包

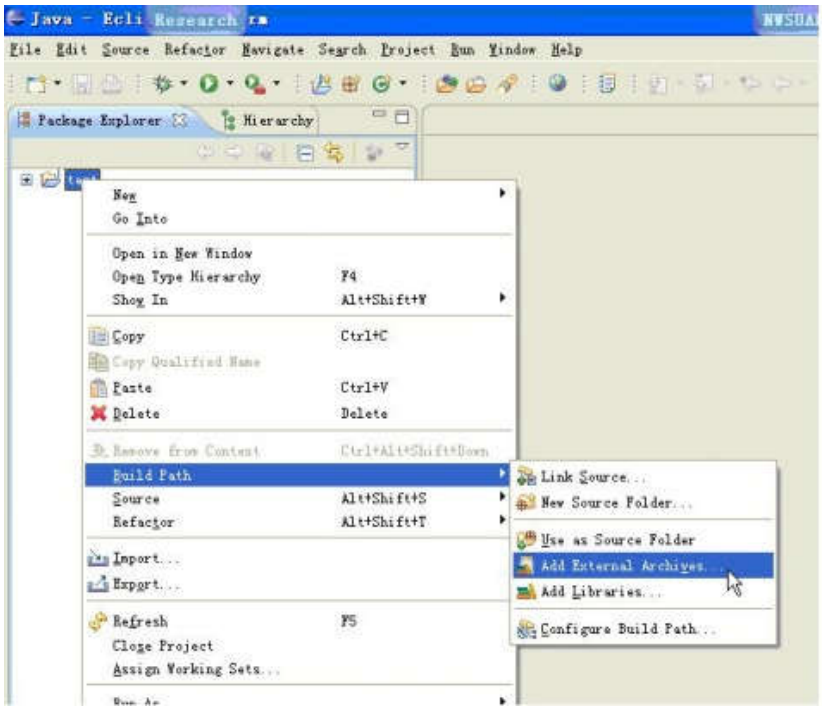
- **用户图形界面下使用 WEKA**

- **Eclipse 下使用 Weka.jar 包**

步骤：（1）首先，新建一个 JAVA 工程，如下图：



（2）接下来，导入 weka. jar, 如下图：



(3）创建一个 package 包

New -〉package -> 输入 cn.uestc.preprocessing

(4) 创建 JAVA 文件

在包下面创建一个新的 class: Test.java
New -> Class -> 输入 Test

（5）编辑 Test.java 文件

1）首先输入需要的包：

```
import weka.core.Attribute;
import weka.core.Instance;
import weka.core.Instances;
import weka.core.converters.ConverterUtils.DataSource;
import weka.filters.Filter;
import weka.filters.unsupervised.attribute.Normalize;
```

2）在主函数里面输入你们想做的事情。比如我们想对数据进行归一化操作。

```
//读取数据
System.out.println("Step 1. 读取数据...");
DataSource source = new DataSource("D:/weka-3-7-7/weka-3-7-7/data/iris.arff");
Instances instances = source.getDataSet();

//归一化
 System.out.println("Step 2. 归一化...");
 Normalize norm = new Normalize();
 norm.setInputFormat(instances);
 Instances newInstances = Filter.useFilter(instances, norm);

//打印或者保存

//打印属性名
int numOfAttributes = newInstances.numAttributes();
for(int i = 0; i < numOfAttributes ;++i)
{
        Attribute attribute = newInstances.attribute(i);
        System.out.print(attribute.name() + "         ");
 }
System.out.println();
//打印实例
int numOfInstance = newInstances.numInstances();
for(int i = 0; i < numOfInstance; ++i)
{
      Instance instance = newInstances.instance(i);
      System.out.print(instance.toString() + "         ");
      System.out.println();
}
//保存归一化后的数据到新文件
DataSink.write("D:/weka-3-7-7/weka-3-7-7/data/iris_norm.arff", newInstances);
```

完整源码：

```java
package cn.uestc.preprocessing;

import weka.core.Attribute;
import weka.core.Instance;
import weka.core.Instances;
import weka.core.converters.ConverterUtils.DataSink;
import weka.core.converters.ConverterUtils.DataSource;
import weka.filters.Filter;
import weka.filters.unsupervised.attribute.Normalize;

public class Test {

    public static void main(String[] args) throws Exception{
        System.out.println("++++++++++Example 1: Normalize Data via weka.++++++++");

        System.out.println("Step 1. 读取数据...");
        DataSource source = new DataSource("D:/weka-3-7-7/weka-3-7-7/data/iris.arff");
        Instances instances = source.getDataSet();


        System.out.println("Step 2. 归一化...");
        Normalize norm = new Normalize();
        norm.setInputFormat(instances);
        Instances newInstances = Filter.useFilter(instances, norm);

        System.out.println("Step 3. 归一化之后的数据（打印）...");

        System.out.println("--------------------------------------");
       try {
           //打印属性名
           int numOfAttributes = newInstances.numAttributes();
           for(int i = 0; i < numOfAttributes ;++i)
           {
               Attribute attribute = newInstances.attribute(i);
               System.out.print(attribute.name() + "     ");
           }
           System.out.println();
           //打印实例
           int numOfInstance = newInstances.numInstances();
           for(int i = 0; i < numOfInstance; ++i)
           {
               Instance instance = newInstances.instance(i);
               System.out.print(instance.toString() + "     ");
               System.out.println();
           }

       } catch (Exception e) {
           e.printStackTrace();
       }
       System.out.println("Step 4. 保存归一化后的数据到新文件...");
       System.out.println("--------------------------------");
       DataSink.write("D:/weka-3-7-7/weka-3-7-7/data/iris_norm.arff", newInstances);
       System.out.println("Congratulations.");
    }

}
```

实验报告：请比较一下 GUI 和自己编程调用归一化函数结果是否不同？截图并放在实验报告中。

1.3 缺失值处理

创建一个新类，实现对 labor 数据缺失值的处理；

策略一：所有完整数据均值 （必做）

策略二：最临近数据的均值/中位数 （可选）

伪代码(策略一）：

double *meanV* = 0; int *n* =0;

for (int *i*=0; *i*<data.length; *i*++){

if(data[*i*][*index*]!=*null*){

***meanV* += data[*i*][*index*];**

***n*++;**

}

}

Specify the missing value as *meanV*/*n*;

完整源码：

```
package cn.uestc.preprocessing;

import weka.core.Instances;
import weka.core.converters.ConverterUtils.DataSink;
import weka.core.converters.ConverterUtils.DataSource;

/**
 * @author Junming Shao
 *
 */
public class MissingValue {

    /**
     * @param args
     * @throws Exception
     */
    public static void main(String[] args) throws Exception {
        System.out.println("++++++++++Example 2: Missing Value Handling.++++++++");

        System.out.println("Step 1. 读取数据...");
        String fn ="D:/weka-3-7-7/weka-3-7-7/data/laborMissing.arff";
        DataSource source = new DataSource(fn);
        Instances instances = source.getDataSet();
        int dim = instances.numAttributes();
        int num = instances.numInstances();
```

```java
        System.out.println("Step 2. 缺失值处理...");
        //计算全局平均值
        double[] meanV = new double[dim];
        for(int i=0;i<dim;i++){
            meanV[i] = 0;
            int count = 0;
            for(int j=0;j<num;j++){
                if(!instances.instance(j).isMissing(i)){
                    meanV[i] +=instances.instance(j).value(i);
                    count++;
                }
            }
            meanV[i] = meanV[i]/count;
        }
                //利用全局平均值代替缺失值
        for(int i=0;i<dim;i++){
            for(int j=0;j<num;j++){
                if(instances.instance(j).isMissing(i)){
                    instances.instance(j).setValue(i, meanV[i]);
                }
            }
        }
        System.out.println("Step 3. 保存数据到新文件...");
        DataSink.write(fn.substring(0, fn.length()-6)+"_handle.arff", instances);
        System.out.println("Finish.");

    }
}
```

实验报告： 将归一化后的数据截图放在实验报告中。

1.4 特征筛选

通过调用 weka 函数进行特征筛选 （GUI 和编程模式均需要）

（1）**GUI 方式：比较不同特征筛选的方法**

实验结果：抓图显示实验结果。

（2）**编程调用方式：**

步骤：

1） **引入需要的类包；**

2） **读取文件；**

3） **创建特征筛选类 AttributeSelection 对象；排序类 Ranker 对象；**

4） **设置相关参数（可以参考 GUI）**

5） **选择特征筛选算法：InfoGainAttributeEval**

6) **保存新数据**

完整源码：

```java
package cn.uestc.preprocessing;

import weka.attributeSelection.ASEvaluation;
import weka.attributeSelection.InfoGainAttributeEval;
import weka.attributeSelection.Ranker;
import weka.core.Instances;
import weka.core.converters.ConverterUtils.DataSink;
import weka.core.converters.ConverterUtils.DataSource;
import weka.filters.Filter;
import weka.filters.supervised.attribute.AttributeSelection;

 * @author Junming Shao
public class FeatureSelection {

     * @param args
    public static void main(String[] args) throws Exception {

        System.out.println("++++++++++Example 2: Feature Selection via weka.++++++++");

        System.out.println("Step 1. 读取数据...");
        String fn ="D:/weka-3-7-7/weka-3-7-7/data/iris.arff";
        DataSource source = new DataSource(fn);
        Instances instances = source.getDataSet();

        System.out.println("Step 2. 特征筛选...");

        //select features from data
        int k =2;
        AttributeSelection as = new AttributeSelection(); //特征筛选类
        Ranker rank = new Ranker();
        rank.setThreshold(0.0); //参数设置，筛选大于某阈值的特征
        rank.setNumToSelect(k); // Number of features want to select

        ASEvaluation ae = new InfoGainAttributeEval(); //change here: different feature selection
        //ASEvaluation ae = new ReliefFAttributeEval();
        as.setEvaluator(ae);
        as.setSearch(rank);
        as.setInputFormat(instances);
        Instances reductData = Filter.useFilter(instances, as);

        System.out.println("Step 3. 保存规约后的数据到新文件...");
        DataSink.write(fn.substring(0, fn.length()-6)+"_reducted.arff", reductData);
        System.out.println("Finish.");


    }
```

实验报告：截图显示实验结果

1.5 数据可视化（可选）

平台：matlab

- 显示数据的直方图（hist 函数）
- 散点图（scatter）等

实验报告：截图显示实验结果

实验二、关联规则挖掘

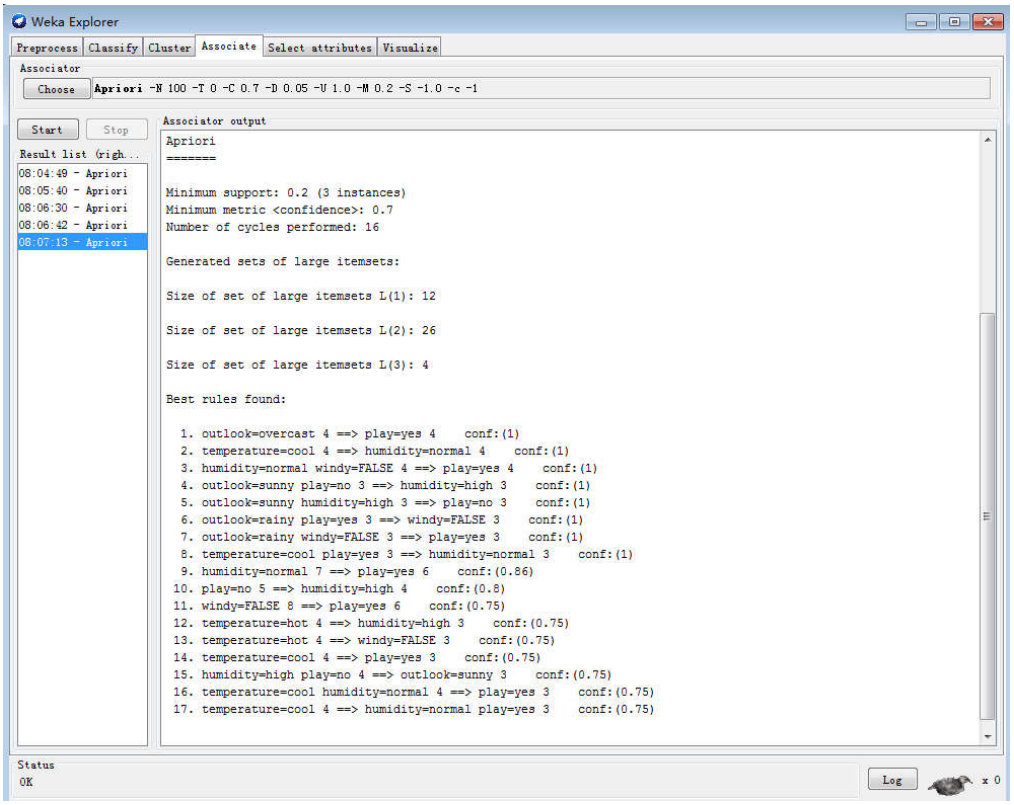
1、实验内容：

（1）学会调用 WEKA 包实现关联规则的挖掘

（2）自己编程实现 Apriori 算法

2、实验步骤

(1) 使用 WEKA 图形界面实现关联规则挖掘



注意：输入文件的属性必须是离散化的。如果非离散化，请首先进行离散化操作！

实验报告：截图显示实验结果

（2） 自己动手实现 Apriori 算法

首先认真理解 Apriori 算法，不是完全清晰的先理清思路。

实现 Apriori 算法的基本流程（参考）：

1）首先根据实际问题，获取数据；

2）在此基础上，对数据进行预处理（如有需要）；

3）建立 Apriori 算法类，名称自定，但建议就直接起名为 Apriori；

4）定义 Apriori 的全局变量，在这个算法中，主要涉及到 min_sup 及 min_conf；

5)在主函数里面开始实现算法

频繁项集生成:

首先，读入数据（事务数据库）；

其次，找出所有的频繁一项集（遍历整个数据集，找出所有一项集，并计算计数，如果 support>min_sup，则将其放入频繁一项集的变量中。改变量可以用 Map<Set<String>, Float>存储项集及对应的支持度）；

在此基础上，构建 K 项集（K>1）

首先是在 K-1 项集的基础上生成 K 项候选集 ，包括自连接步和剪枝步。 其中自连接步将任意两个项集如果 K-2 项相同，剩余的一项不同的两个项集连接。可以考虑用 JAVA 中的集合操作运算实现。

剪枝步就是将自连接生成的候选集的任意子集在 K-1 项中是否存在，否则删除 （实现 has_infrequent_subset 函数）。

产生获选 K 项集后，遍历整个数据集，检查这些候选集是否频繁，如果是，加入 K 项频繁项集中。循环这个过程，直到找到的 K 频繁项集为空，退出。

关联规则获取：在找到频繁项集的基础上,将 K>1 的任意频繁项集分成任意的两子集 A 和 B（直接调用 ProperSubsetCombination， 当然也可以自己写）， 然后计算 A->B 及 B->A 的置信度。如果置信度>min_conf，那么这些规则是强规则，并保存起来。

源码参考:

(1) 引入包及构造函数

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
import java.util.TreeMap;

public class AprioriAlgorithm {
    private Map<Integer, Set<String>> txDatabase; // 事务数据库
    private Float minSup; // 最小支持度
    private Float minConf; // 最小置信度
    private Integer txDatabaseCount; // 事务数据库中的事务数

    private Map<Integer, Set<Set<String>>> freqItemSet; // 频繁项集集合
    private Map<Set<String>, Set<Set<String>>> assiciationRules; // 频繁关联规则集合

    public AprioriAlgorithm(Map<Integer, Set<String>> txDatabase, Float minSup,
            Float minConf)
    {
        this.txDatabase = txDatabase;
        this.minSup = minSup;
        this.minConf = minConf;
        this.txDatabaseCount = this.txDatabase.size();
        freqItemSet = new TreeMap<Integer, Set<Set<String>>>();
        assiciationRules = new HashMap<Set<String>, Set<Set<String>>>();
    }
```

(2) 主函数

```java
String fn = "C:/Users/thinkpad/Desktop/AssRule/AssRule/data/test.txt";
float minSup = 0.5f;
float minConf = 0.6f;
File file = new File(fn);
FileReader fr = new FileReader(file);
BufferedReader br = new BufferedReader(fr);
Map<Integer, Set<String>> DB = new HashMap<Integer, Set<String>>();

String line;
String sp = ","; //分隔符
int num = 0;
while ((line = br.readLine()) != null){
    String[] temp = line.trim().split(sp);
    Set<String> set = new TreeSet<String>();
    for (int i = 1; i < temp.length; i++){ //第一列为ID，不读入
        set.add(temp[i].trim());
    }
    num++;
    DB.put(num,set);
}


    //构建类对象
    Apriori  apr = new Apriori(DB,minSup,minConf);
    apr.findAllFreqItemSet();
    apr.findAssociationRules();
```

（3）寻找频繁项集

```java
public void findAllFreqItemSet(){
    // 频繁1-项集
    Map<Set<String>, Float> freqOneItemSet = this.find_Frequent_One_Itemsets();
    freqItemSets.put(1, freqOneItemSet);
    System.out.println("频繁1-项集:" + freqOneItemSet);

    // 频繁k-项集(k>1)
    int k = 2;
    while(true){
        Set<Set<String>> candFreItemsets = apriori_Gen(k, freqItemSets.get(k-1).keySet());
        Map<Set<String>, Float> freqKItemSetMap = getFreqKItemSet(k,candFreItemsets);
        if (!freqKItemSetMap.isEmpty()){
            freqItemSets.put(k, freqKItemSetMap);
        } else{
            break;
        }
        System.out.println("频繁"+k+"-项集:" + freqKItemSetMap);
        k++;
    }
}
```

find_Frequent_One_Itemsets 函数

```java
//频繁一项集函数
public Map<Set<String>, Float> find_Frequent_One_Itemsets(){

    Map<Set<String>, Float> L1 = new HashMap<Set<String>, Float>();
    Map<Set<String>, Integer> item1SetMap = new HashMap<Set<String>, Integer>();
    Iterator<Map.Entry<Integer, Set<String>>> it = DB.entrySet().iterator();
    // 支持度计数，生成候选频繁1-项集
    while (it.hasNext()){
        Map.Entry<Integer, Set<String>> entry = it.next();
        Set<String> itemSet = entry.getValue();
        for (String item : itemSet){
            Set<String> key = new HashSet<String>();
            key.add(item.trim());
            if (!item1SetMap.containsKey(key)){
                item1SetMap.put(key, 1);
            } else{
                int value = 1 + item1SetMap.get(key);
                item1SetMap.put(key, value);
            }
        }
    }

    //找出支持度大于minSup的频繁一项集
    Iterator<Map.Entry<Set<String>, Integer>> iter = item1SetMap.entrySet().iterator();
    while (iter.hasNext())
    {
        Map.Entry<Set<String>, Integer> entry = iter.next();
        // 计算支持度
        Float support = new Float(entry.getValue().toString())/new Float(num);
        if (support >= minSup)
            L1.put(entry.getKey(), support);
    }
    return L1;
}
```

apriori_Gen 函数

```java
//从K项频繁项集产生候选K+1项集函数
public Set<Set<String>> apriori_Gen(int k, Set<Set<String>> freqKItemSet){
    Set<Set<String>> candFreqKItemSet = new HashSet<Set<String>>();
    Iterator<Set<String>> it1 = freqKItemSet.iterator();
    while (it1.hasNext()){
        Set<String> itemSet1 = it1.next();
        Iterator<Set<String>> it2 = freqKItemSet.iterator();
        while (it2.hasNext()){
            Set<String> itemSet2 = it2.next();
            if(!itemSet1.equals(itemSet2)){
                // 连接步：k-项集中前k-2个项必须相同
                Set<String> commItems = new HashSet<String>();
                commItems.addAll(itemSet1);
                commItems.retainAll(itemSet2);
                if (commItems.size() == k-2){
                    Set<String> candiItems = new HashSet<String>();
                    candiItems.addAll(itemSet1);
                    candiItems.removeAll(itemSet2);
                    candiItems.addAll(itemSet2);
                    //剪枝步：查看生成的K项集的任意K-1项集是否都在已获取的频繁K-1项 freqKItemSet中
                    if(!has_infrequent_subset(candiItems,freqKItemSet)){
                        candFreqKItemSet.add(candiItems);
                    }

                    }
                }
            }
        }
        return candFreqKItemSet;
    }
```

has_infrequent_subset 函数

```java
//判断一个候选是否应该剪枝
private boolean has_infrequent_subset(Set<String> itemSet,Set<Set<String>> freqKItemSet){
    //获取itemSet的所有k-1子集
    Set<Set<String>> subItemSet = new HashSet<Set<String>>();
    Iterator<String> itr = itemSet.iterator();
    while (itr.hasNext()){
        //深拷贝
        Set<String> subItem = new HashSet<String>();
        Iterator<String> it = itemSet.iterator();
        while(it.hasNext()){
            subItem.add(it.next());
        }
        //去掉一个项后即为k-1子集
        subItem.remove(itr.next());
        subItemSet.add(subItem);
    }
    Iterator<Set<String>> it = subItemSet.iterator();
    while (it.hasNext()){
        if(!freqKItemSet.contains(it.next()))
            return true;
    }
    return false;
}
```

getFreqKItemSet 函数

```java
public Map<Set<String>, Float> getFreqKItemSet(int k,Set<Set<String>> candFreqKItemSet){

    Map<Set<String>, Integer> candFreqKItemSetMap = new HashMap<Set<String>, Integer>();
    // 扫描事务数据库
    Iterator<Map.Entry<Integer, Set<String>>> it = DB.entrySet().iterator();
    // 统计支持数

    while (it.hasNext()){
        Map.Entry<Integer, Set<String>> entry = it.next();
        Iterator<Set<String>> iter = candFreqKItemSet.iterator();
        while (iter.hasNext()){
            Set<String> s = iter.next();
            if(entry.getValue().containsAll(s)){
                if(!candFreqKItemSetMap.containsKey(s)){
                    candFreqKItemSetMap.put(s, 1);
                }else{
                    int value = 1 + candFreqKItemSetMap.get(s);
                    candFreqKItemSetMap.put(s,value);
                }
            }
        }
    }

    //System.out.println(candFreqKItemSetMap.size());
    // 计算支持度并生成最终的频繁k-项集
    Map<Set<String>, Float> freqKItemSetMap = new HashMap<Set<String>, Float>();
    Iterator<Map.Entry<Set<String>, Integer>> itr = candFreqKItemSetMap.entrySet().iterator();
    while (itr.hasNext()){
        Map.Entry<Set<String>, Integer> entry = itr.next();
        // 计算支持度
        float support = new Float(entry.getValue().toString())/num;
        if (support < minSup){ // 如果不满足最小支持度，删除
            itr.remove();
        } else{
            freqKItemSetMap.put(entry.getKey(), support);
        }
    }

    return freqKItemSetMap;
}
```

（3）从频繁项集中找出强关联规则：findAssociationRules

```java
public void findAssociationRules(){
    //freqItemSets.remove(1); // 删除频繁1-项集
    Iterator<Map.Entry<Integer, Map<Set<String>, Float>>> it = freqItemSets.entrySet().iterator();
    while (it.hasNext()){
        Map.Entry<Integer, Map<Set<String>, Float>> entry = it.next();
        for (Set<String> itemSet : entry.getValue().keySet()){
            // 对每个频繁项集进行关联规则的挖掘
            int n = itemSet.size() / 2; // 根据集合的对称性，只需要得到一半的真子集
            for (int i = 1; i <= n; i++){
                // 得到频繁项集元素itemSet的作为条件的真子集集合
                Set<Set<String>> subset = ProperSubsetCombination.getProperSubset(i, itemSet);
                // 对条件的真子集集合中的每个条件项集，获取到对应的结论项集，从而进一步挖掘频繁关联规则
                for (Set<String> conditionSet : subset)
                {
                    Set<String> conclusionSet = new HashSet<String>();
                    conclusionSet.addAll(itemSet);
                    conclusionSet.removeAll(conditionSet); // 删除条件中存在的频繁项

                    int s1 = conditionSet.size();
                    int s2 = conclusionSet.size();
```

```java
                    float sup1 = freqItemSets.get(s1).get(conditionSet);
                    float sup2 = freqItemSets.get(s2).get(conclusionSet);
                    float sup = freqItemSets.get(s1+s2).get(itemSet);

                    float conf1 = sup/sup1;
                    float conf2 = sup/sup2;
                    if (conf1 >= minConf){
                        if (assiciationRules.get(conditionSet) == null){ // 如果不存在以该条件频繁项集为条件的关联规则
                            Set<Map<Set<String>,Float>> conclusionSetSet = new HashSet<Map<Set<String>,Float>>();
                            Map<Set<String>, Float> sets = new HashMap<Set<String>, Float>();
                            sets.put(conclusionSet, conf1);
                            conclusionSetSet.add(sets);
                            assiciationRules.put(conditionSet, conclusionSetSet);
                        } else
                        {
                            Map<Set<String>, Float> sets = new HashMap<Set<String>, Float>();
                            sets.put(conclusionSet, conf1);
                            assiciationRules.get(conditionSet).add(sets);
                        }
                    }

                    if (conf2 >= minConf)
                    {
                        if (assiciationRules.get(conclusionSet) == null)
                        { // 如果不存在以该结论频繁项集为条件的关联规则
                            Set<Map<Set<String>,Float>> conclusionSetSet =
                                    new HashSet<Map<Set<String>,Float>>();
                            Map<Set<String>, Float> sets = new HashMap<Set<String>, Float>();
                            sets.put(conclusionSet, conf2);
                            conclusionSetSet.add(sets);
                            assiciationRules.put(conclusionSet, conclusionSetSet);
                        } else
                        {
                            Map<Set<String>, Float> sets = new HashMap<Set<String>, Float>();
                            sets.put(conditionSet, conf2);
                            assiciationRules.get(conclusionSet).add(sets);
                        }
                    }
                }
            }
        }
    }
    System.out.println("关联规则(强规则) : " + assiciationRules);
}
```

将关键代码及运行结果放在实验报告中。

实验三、分类算法

1、实验内容

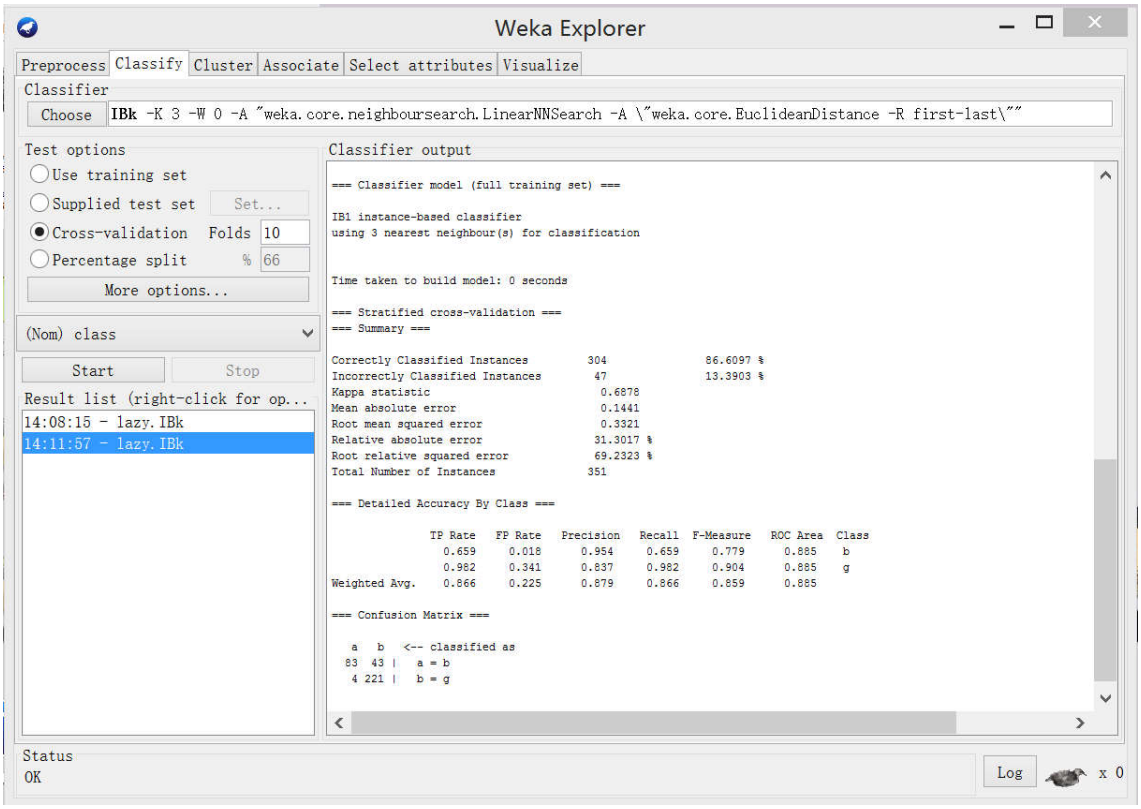
（1）实现 KNN 算法

（2）初步学习 MATLAB 实现 ANN 算法

（3）实现决策树算法（可选或课下）

2、KNN 算法

(1) 使用 WEKA 图形界面实现 KNN



注意：输入文件的属性必须是数值型的。

实验报告：截图显示实验结果

（2） 自己动手实现 KNN 算法

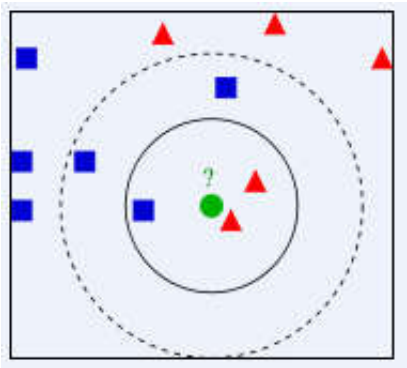
KNN 算法原理

KNN 属于 lazy learning，不会对训练样本数据进行学习，其做法是：对于一个新数据，计算它与训练集中数据的距离，选择最短的 k 个作为邻居，然后预测它的类别和 k 个邻居中其所属类别最多的一致。

KNN 算法优点：简单，快速，准确率较高

KNN 算法缺点：

（1）K 值的选择



（2）训练数据

由于 KNN 属于 lazy learning，不会对训练数据集做深入学习，而是单纯的考虑数据之间的距离，所以 KNN 的预测效果强依赖于训练数据。KNN 的不足体现在：

A. 训练样本能否正确体现数据真实分布

比如，训练样本是否大多属于某一个类，或某几个类；

B. 训练数据是否属于同一分布

比如，对噪声数据敏感

（3）其他

比如距离的选定，距离的计算复杂性

三个步骤: (1）算距离（2)找邻居（3）投票分类

源码参考：

（1）定义一个内部类 Data，用于存取封装数据

```
public class KnnClassifier {
    private class Data {
        double[] data;
        String label;

        Data(double[] data, String label)
        {
            this.data = data;
            this.label = label;
        }
    }
```

（2）算法主类

算法 main 主函数入口，然后执行三块，分别是读入训练数据，预测新数据和预测结果输出

```
ArrayList<Data> trainSet;
ArrayList<Data> testSet;
int k;

public static void main(String args[])
{
    KnnClassifier knn = new KnnClassifier("data/train.txt",1);
    knn.predict("data/test.txt");

}
public KnnClassifier(String filePath, int k)
{
    trainSet=loadDataSet(filePath);

    this.k = k;
}
```

训练数据读入函数

```
public ArrayList<Data> loadDataSet(String filePath)
{
    ArrayList<Data> dataSet = new ArrayList<Data>();
    File file = new File(filePath);
    FileReader fr;
    try
    {
        fr = new FileReader(file);
        BufferedReader bis = new BufferedReader(fr);
        String line = null;
        while ((line = bis.readLine()) != null)
        {
            String[] str = line.trim().split(",");
            double[] data = new double[str.length - 1];
            for (int i = 0; i < data.length; i++)
            {
                data[i] = Double.parseDouble(str[i]);
            }
            Data dataObject = new Data(data, str[str.length - 1]);
            dataSet.add(dataObject);
        }
        bis.close();

    } catch (Exception e)
    {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    return dataSet;

}
```

对新数据进行预测，首先读入新数据，然后调用 classify 函数进行预测

```java
public void predict(String testPath){
    testSet = this.loadDataSet(testPath);
    int correct = 0;
    for(int n=0;n<testSet.size();n++){
        Data data = testSet.get(n);
        String predict = classify(data);
            System.out.println("预测label："+predict+" 真实label " + data.label);
            if(predict.equals(data.label)){
                correct++;
            }
    }
    System.out.println("准确率:"+correct*1.0/testSet.size());

}
```

预测函数。根据新数据与训练数据之间的距离进行预测，用 getDis 函数计算欧氏距离

```java
public String classify(Data test)
{
    double[] distances = new double[trainSet.size()];
    //获取测试点离训练数据的距离
    for (int n = 0; n < trainSet.size(); n++)
    {
        distances[n] = getDist(test, trainSet.get(n));
    }
    HashMap<String, Integer> voteMap = new HashMap<String, Integer>();
    //循环k次，取得前k最小距离，进行投票
    for (int n = 0; n < k; n++)
    {
        double min = Integer.MAX_VALUE;
        int minIndex = -1;
        for (int m = 0; m < trainSet.size(); m++)
        {
            if (distances[m] < min)
            {
                min = distances[m];
                minIndex = m;
            }
        }
```

```
        if (minIndex == -1)
        {
            System.out.println("error");
            System.exit(0);
        }
        distances[minIndex] = Integer.MAX_VALUE;
        if (voteMap.containsKey(trainSet.get(minIndex).label))
        {
            voteMap.put(trainSet.get(minIndex).label,
                    voteMap.get(trainSet.get(minIndex).label) + 1);
        } else
        {
            voteMap.put(trainSet.get(minIndex).label, 1);
        }
    }
    //投票结果的计算
    Iterator<String> i = voteMap.keySet().iterator();
    String predict = null;
    int maxVote = 0;
    while (i.hasNext())
    {
        String key = i.next();
        if (voteMap.get(key) > maxVote)
        {
            maxVote = voteMap.get(key);
            predict = key;
        }
    }
    return predict;
}
```

```
private double getDist(Data test, Data data)
{
    double sum = 0;
    for (int n = 0; n < test.data.length; n++)
    {
        sum =  (test.data[n]-data.data[n])*(test.data[n]-data.data[n]);
    }
    return Math.sqrt(sum);
}
```

将关键代码及运行结果放在实验报告中。

3、尝试利用 matlab 实现人工神经网络，包括感知机和后向传播网络（BP）

（1）感知机

```
%% 感知机测试

close all; clc; clear all;

%%P为输入向量;        %T为目标向量
P=[-0.5 -0.5 0.3 0 ;
   -0.5 0.5 -0.5 1];
T=[1 1  0 0];
plotpv(P,T);          %  绘制输入向量

%初始化一个网络
net = newp([-1 1; 0 1], 1);
net = init(net);
y = sim(net, P);
e = T- y;
w = net.iw{1,1}
b = net.b{1}
```

```
plotpc(w,b);
while (mae(e)>0.0015)
  dw=learnp(w,P,[],[],[],[],e,[],[],[],[],[]);
  db=learnp(b,ones(1,4),[],[],[],[],e,[],[],[],[],[]);
  %调整权值矩阵和阈值矩阵
  w=w+dw
  b=b+db
  net.iw{1,1}=w;
  net.b{1}=b;
  plotpc(w,b)
  pause;
  y=sim(net,P)
  e=T-y;
end
disp('运行结束！')
```

（2）BP 网络

```
clear all; close all; clc;  warning off;
p = [0 1 2 3 4 5 6 7 8];
t = [0 0.84 0.91 0.14 -0.77 -0.96 -0.28 0.66 0.99];
%建立一个前向网络，隐层为10，输出为1个单元，训练函数为trainlm
net = newff([0 8],[10 1],{'tansig' 'purelin'},'trainlm');
y1 = sim(net,p);  %仿真
plot(p,t,'o',p,y1,'x');
%设置训练参数
net.trainParam.epochs = 50;
net.trainParam.goal = 0.01;
%训练网络
net = train(net,p,t);
%仿真
y2 = sim(net,p);
%% test
test = 6.5; y3 = sim(net,test)
plot(p,t,'o',p,y1,'x',p,y2,'*');
```

4、决策树分类　（可选）

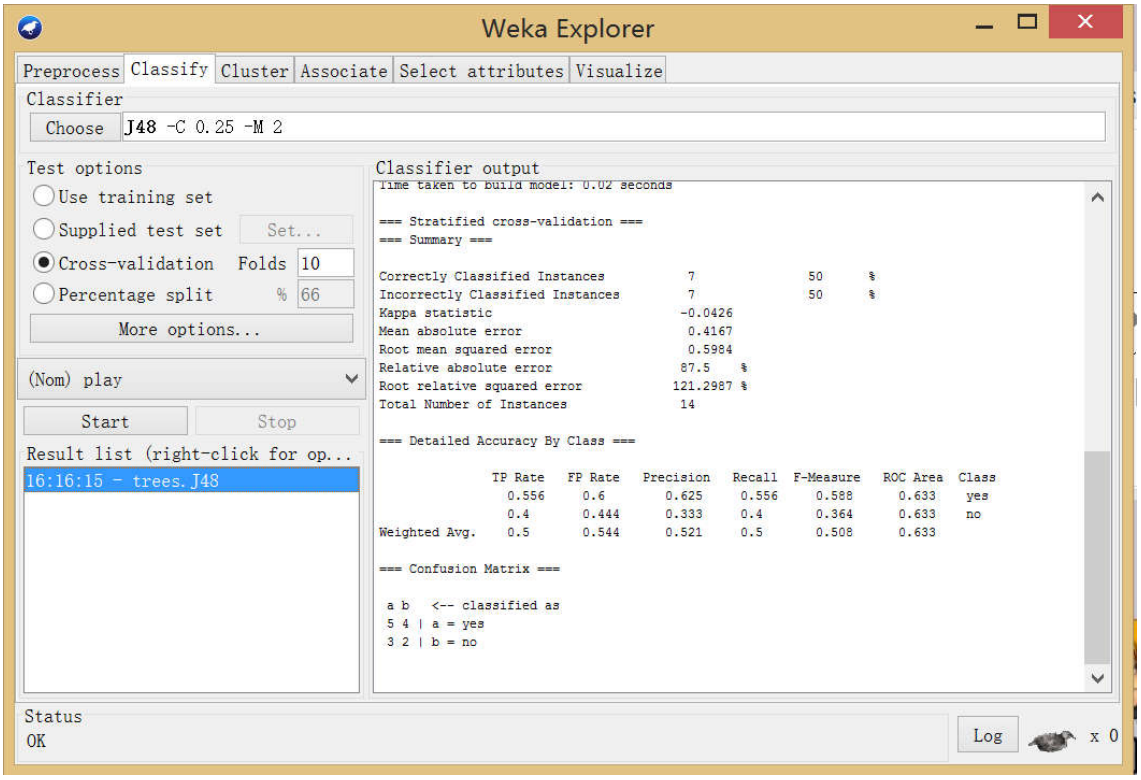
4.1、实验内容

（1）学会调用 WEKA 包处理决策树分类问题

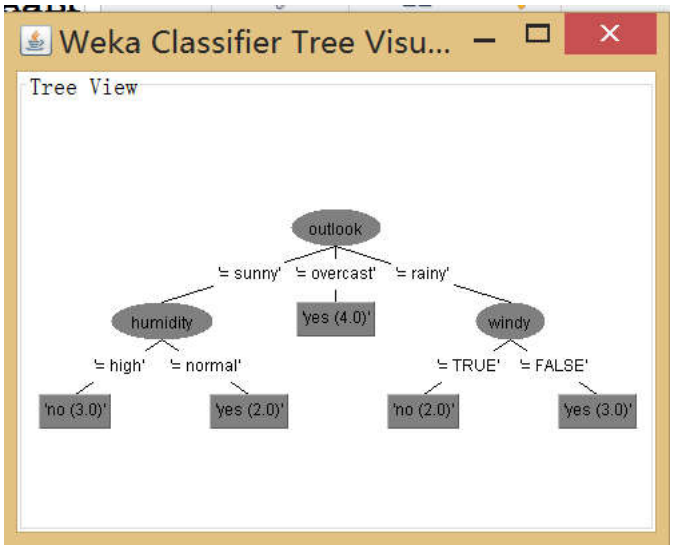
（2）自己编程实现决策树算法

4.2、实验步骤

(1) 使用 WEKA 图形界面实现决策树



在上方界面左下角的 Result list 中，选中本次执行的任务，鼠标右键，选择 Visualize tree，可以显示结果

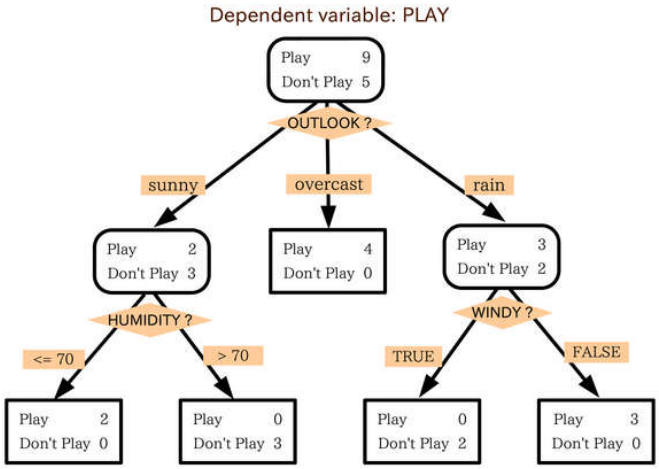


实验报告：截图显示实验结果

（2） 自己动手实现决策树算法

2.1 决策树算法原理

1）什么是决策树



类似于流程图的树结构，每个分枝代表该测试的输出，每个树叶节点代表类或类分布

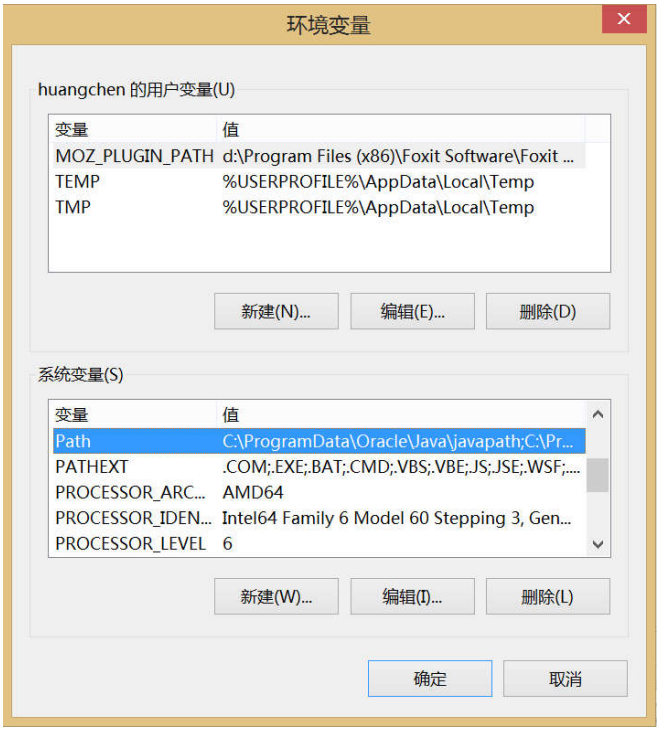
2）决策树的构造

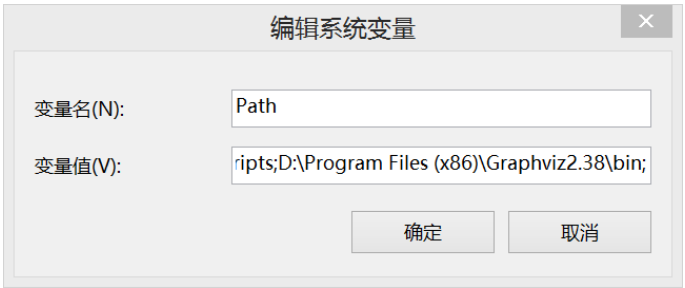
- 核心思想在于决策树越向下生长，使节点的纯度越大。
- 自顶向下的分治方式构造决策树
- 递归的通过选择相应的测试属性，通过选择最有分类能力的属性作为决策树的当前结点，由此划分样本，其中测试属性是根据某种启发信息或者是统计信息来进行选择，如信息增益

源码参考：

NOTE：决策树算法采用 XML 文件保存训练模型结果，即树形结果，通过 Graphviz 软件进行可视化。因此为处理 XML 文件，需要引入 dom4j 和 jaxen 两个 jar 包；（如果

不需要可视化，可以不做），为可视化树形结果需要安装 Graphviz 软件（比如安装路径是 D:\Program Files (x86)\Graphviz2.38），然后配置环境变量（我的电脑--属性--高级系统设置--环境变量--系统变量中找到 Path，双击，在变量值中**添加** D:\Program Files (x86)\Graphviz2.38\bin;　注意添加前用英文分号隔开）

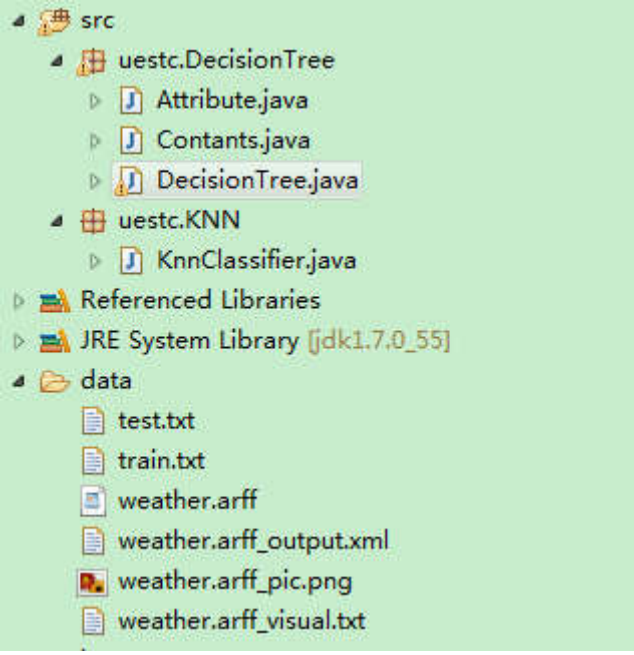




根据 Graphviz 安装路径修改 Contants 的 PIC_UTIL_PATH 变量

```java
public class Contants {
    //Graphviz2.38安装目录
    public static final String PIC_UTIL_PATH = "D:/Program Files (x86)/Graphviz2.38/bin/dot.exe ";
    //训练数据
    public static final String DATA_PATH = "data\\weather.arff";
    //模型输出路径
    public static final String OUTPUT_PATH = DATA_PATH + "_output.xml";
//    public static final String LABEL_MARK = "#";
```

运行程序，得到结果，结果图保存在 data 目录下。(**实验报告：截图显示实验结果**)



实验完毕。

实验四、聚类

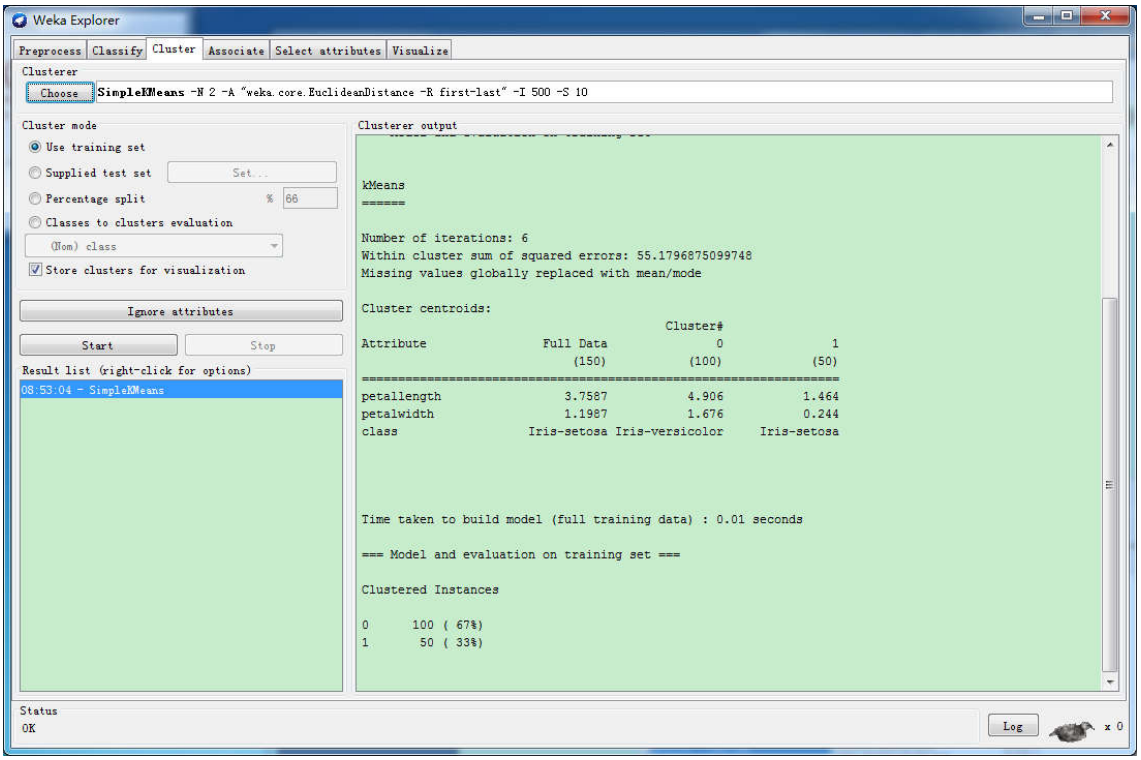
1、实验内容

（1）学会调用 WEKA 包和自己编程实现 kmeans 聚类算法；

（2）学会调用 WEKA 包和自己编程实现 DBSCAN 聚类算法；

2、实验步骤

(1) 使用 WEKA 图形界面实现 Kmeans



实验报告：截图显示实验结果

(2) 自己动手实现 Kmeans 算法

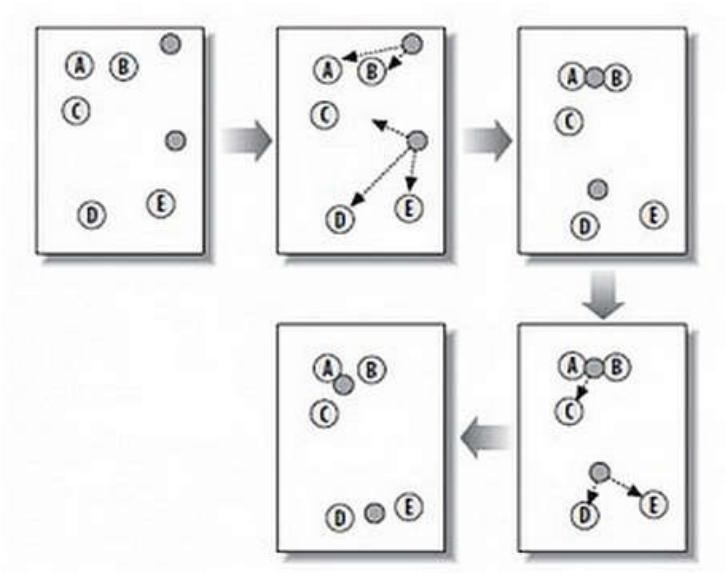
2.1 Kmeans 算法原理

K-Means 的算法如下：

（1）随机在图中取 K（如图 K=2）个聚类中心。

（2）然后对图中的所有点求到这 K 个种子点的距离，假如点 Pi 离聚类中心 Si 最近，那么 Pi 属于 Si 点群。接下来，我们要移动聚类中心到属于他的“点群”的中心。

（3）然后重复第 2）和第 3）步，直到聚类中心几乎不发生变化。



K-Means 主要有两个最重大的缺陷——都和初始值有关：

（1）K 是事先给定的，这个 K 值的选定是非常难以估计的。很多时候，事先并不知道给定的数据集应该分成多少个类别才最合适。

（2）K-Means 算法需要用初始随机种子点来搞，这个随机种子点太重要，不同的随机种子点会有得到完全不同的结果。

2.2 源码参考:

(1) 定义一个主类 Kmeans

```java
import java.io.File;
import java.io.FileReader;
import java.util.ArrayList;
import java.util.Random;

import util.PicUtility;

public class Kmeans {

    ArrayList<double[]> dataSet;
    int clusterNum;
    int dim ; //data dimension

    Kmeans(int clusterNum)
    {
        this.clusterNum = clusterNum;
        dataSet = new ArrayList<double[]>();
    }
```

（3）算法主类

算法 main 主函数入口，然后执行三块，分别是读入数据，聚类，可视化。可视化部分所需要用到 jfreechart 库，所以需要两个库文件：jcommon-1.0.23.jar，jfreechart-1.0.19.jar，把它们引入工程。还有一个 PicUtility.java 文件用于提供方便的可视化接口，把包含它的 util 文件夹 copy 到 src 目录即可。

```java
    public static void main(String args[])
    {
        Kmeans kmean = new Kmeans(3);
        kmean.loadDataSet("dataSet/data");
        kmean.cluster();
    }
```

聚类数据读入：

```
// data load
  public void loadDataSet(String filePath)
  {
    File file = new File(filePath);
    FileReader fr;
    try
    {
      fr = new FileReader(file);
      BufferedReader bis = new BufferedReader(fr);
      String line = null;
      while ((line = bis.readLine()) != null)
      {
        String[] str = line.trim().split(" ");
        double[] data = new double[str.length];
        for (int i = 0; i < data.length; i++)
        {
          data[i] = Double.parseDouble(str[i]);
        }
        dataSet.add(data);
      }
      dim = dataSet.get(0).length;
      bis.close();
    } catch (Exception e)
    {
      e.printStackTrace();
    }

  }
```

聚类:

```
public void cluster()
{
    Random rand = new Random();
    double[][] clusterMeans = new double[clusterNum][dim];
    // random initalize mean of cluster
    for (int n = 0; n < clusterNum; n++)
    {
        double[] data = new double[dim];
        for (int m = 0; m < dim; m++)
        {
            data[m] = rand.nextDouble() * 100;
        }
        clusterMeans[n] = data;
    }

    boolean isContinue = true;
    while (isContinue)
    {
        isContinue = false;
        double[][] nextClusterMeans = new double[clusterNum][dim];
        int[] clusterDataNum = new int[clusterNum];
        // assign cluster mean to every data
        for (int n = 0; n < dataSet.size(); n++)
        {
            double minDis = Double.MAX_VALUE;
            int whoCluster = -1;
            for (int m = 0; m < clusterNum; m++)
            {
```

```
            double distance = this.getDist(clusterMeans[m],
                    dataSet.get(n));
            if (distance < minDis)
            {
                whoCluster = m;
                minDis = distance;
            }
        }
        clusterDataNum[whoCluster]++;
        for (int i = 0; i < dim; i++)
        {
            nextClusterMeans[whoCluster][i] += dataSet.get(n)[i];
        }
    }
```

Update the means:

```
//update cluster means
    for (int i = 0; i < clusterNum; i++)
    {
        for (int j = 0; j < dim; j++)
        {
            if(clusterDataNum[i] != 0)
                nextClusterMeans[i][j] /= clusterDataNum[i];
            else
                nextClusterMeans[i][j] = Math.random()*100;
        }
    }
//if there is no big difference between nextClusterMeans and clusterMeans, stop looping
for (int i = 0; i < clusterNum; i++)
{
    if (this.getDist(nextClusterMeans[i], clusterMeans[i]) !=0)
    {
        isContinue = true;
    }
}
clusterMeans = nextClusterMeans;
```

接下来时可视化部分：

```
//visualization
ArrayList<ArrayList<double[]>> clusters = new ArrayList<ArrayList<double[]>>();
for(int n=0;n<clusterNum;n++){
    clusters.add(new ArrayList<double[]>());
}
for (int n = 0; n < dataSet.size(); n++)
{
    double minDis = Double.MAX_VALUE;
    int whoCluster = -1;
    for (int m = 0; m < clusterNum; m++)
    {
        double distance = this.getDist(clusterMeans[m],
                dataSet.get(n));
        if (distance < minDis)
        {
            whoCluster = m;
            minDis = distance;
        }
    }
    clusters.get(whoCluster).add(dataSet.get(n));
}
double[][][] datas = new double[clusterNum][][];
for(int n=0;n<clusterNum;n++){
```

```
            double[][] cluster = new double[clusters.get(n).size()][];
            for(int m=0;m<cluster.length;m++){
                cluster[m] = clusters.get(n).get(m);
            }
            datas[n] = cluster;

        }
        System.out.println("cluster mean:");
        for(int n=0;n<clusterMeans.length;n++){
            for(double x : clusterMeans[n]){
                System.out.print(x+" ");
            }
            System.out.println();
        }

            PicUtility.show(datas, clusterNum);
```

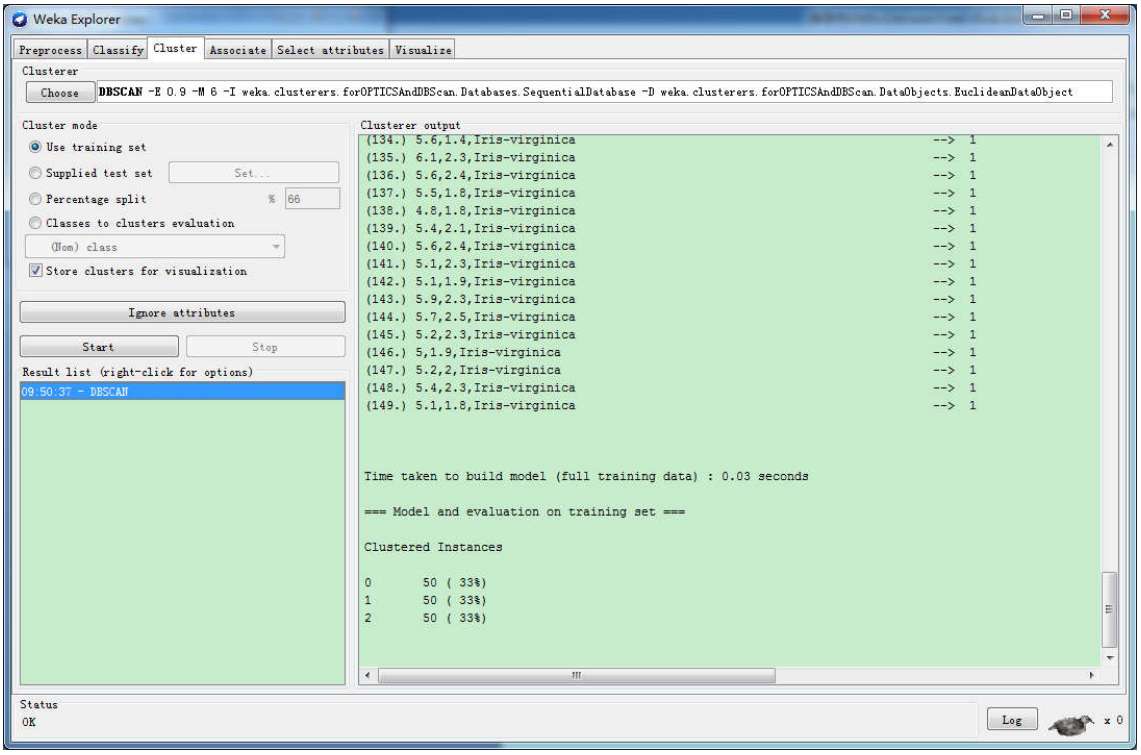
计算距离时，用到的 getDis 函数计算欧氏距离：

```
private double getDist(double[] test, double[] data)
{
    double sum = 0;
    for (int n = 0; n < test.length; n++)
    {
        sum += (test[n] - data[n]) * (test[n] - data[n]);
    }
    return Math.sqrt(sum);
}
```

运行得到实验结果图，

将关键代码及运行结果放在实验报告中。

(3) 使用 WEKA 图形界面实现 DBSCAN



(4)自己实现 DBSCAN，参考源码已给出。